



TALLER PRÁCTICO

Guía Metodológica — Muestreo Multimodal

Capa 3 (voz y rostro) · Tesis CFH

Julián Zuluaga — Docente

9 de junio de 2026

Guía Metodológica — Muestreo de la Capa Multimodal (CFH)

Para: Mireya Camacho • Dirección: Julián Zuluaga • Tesis: Hermenéutica Forense Computacional

Guía operativa y acotada. El objetivo NO es montar un sistema complejo, sino **garantizar que cada muestra de rostro y de voz pertenece al compareciente correcto**, descartar el material inservible, y agregar bien. Con eso, la Capa 3 pasa de captura oportunista a muestra verificada — que es lo que la vuelve defendible.

1. Qué problema resuelve esta guía

El muestreo actual de la Capa 3 tiene un defecto de raíz: **no verifica de quién es la cara ni la voz**.

- **Rostro:** el script toma «la primera cara que detecta» en cada segmento (`calcular_australia_caribe_v2.py`). Esa cara puede ser un magistrado, una víctima o alguien del público.
- **Voz:** se asume que el compareciente es «el SPEAKER que más habla», sin confirmarlo, sobre una diarización sobre-segmentada (en Costa Caribe detectó 62 hablantes).
- **Unidad equivocada:** se trata cada frame como observación, cuando los frames de un mismo gesto están correlacionados.

Resultado: las features no son atribuibles con seguridad al sujeto, y el ICM sale inestable (salta entre versiones). Esto es **muestreo por conveniencia**, el diseño más débil posible.

Lo que esta guía implementa (núcleo mínimo): tres filtros + una corrección de agregación.

#	Filtro	Garantiza
1		que la cara medida es la del compareciente

#	Filtro	Garantiza
	Identidad facial (enrolamiento + verificación)	
2	Calidad mínima (tamaño y frontalidad)	que la cara es utilizable (no ruido de plano abierto)
3	Identidad de voz (verificación manual del hablante)	que la voz medida es la del compareciente
+	Agregación por turno (mediana)	que la unidad es la intervención, no el frame

2. Principio rector (leer antes de codificar)

1. **La unidad de análisis es la intervención/turno del compareciente, no el frame.** Un turno = un tramo continuo donde el compareciente habla. De cada turno sale UNA observación (la mediana de sus frames válidos), no decenas.
2. **No se muestrea: se censa lo válido.** El cómputo (MediaPipe/OpenFace, eGeMAPS) es barato, así que procesa todos los turnos válidos del compareciente. El muestreo solo se justifica cuando computar es caro; aquí no lo es.
3. **Filtrar primero, medir después.** Nunca calcular AUs sobre una cara que no pasó identidad + calidad.

3. Filtro 1 — Identidad facial (el más importante)

La idea: tener **una foto de referencia del compareciente** y, en cada frame, aceptar la cara **solo si se parece** a esa referencia.

3.1 Paso previo (lo hace Mireya, una vez por subcaso)

Obtener **1 foto de referencia buena** del compareciente:

- Buscar en el video un momento donde **sabes con certeza** que es él (cuando aparece su nombre en el banner, o un primer plano claro al presentarse).
- Tomar un **screenshot frontal, nítido, cara grande** (no de perfil, sin oclusión).
- Guardarlo como `ref_<subcaso>.jpg` (ej. `ref_casanare.jpg`).

Una sola foto buena basta. Si quieres más robustez, 2-3 fotos del mismo señor en distintos momentos.

3.2 Código (librería `face_recognition`, sin GPU)

```
pip install face_recognition # incluye dlib; en Colab funciona directo
```

```
import face_recognition, cv2, numpy as np

# 1) Enrolar la referencia (una vez por subcaso)
ref_img = face_recognition.load_image_file("ref_casanare.jpg")
ref_enc = face_recognition.face_encodings(ref_img)[0] # vector de 128 dims

def cara_del_compareciente(frame_bgr, tolerancia=0.55):
    """Devuelve (bbox, True) si la cara que mejor coincide es el compareciente."""
    rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
    locs = face_recognition.face_locations(rgb, model="hog") # hog = rápido, CPU
    if not locs:
        return None, False
    encs = face_recognition.face_encodings(rgb, locs)
    dists = face_recognition.face_distance([ref_enc], *zip(encs)) if False else \
        [face_recognition.face_distance([ref_enc], e)[0] for e in encs]
    i = int(np.argmin(dists))
    es_el = dists[i] <= tolerancia # umbral: 0.50-0.60
    return locs[i], es_el # locs[i] = (top, right, bottom, left)
```

Parámetro clave — `tolerancia`: el default de `face_recognition` es 0.6. Para ser estrictos (preferible aquí), usar **0.50-0.55**. Más bajo = más estricto (menos falsos positivos, pero descarta más). Documenta el valor que uses.

4. Filtro 2 — Calidad mínima del rostro

Una cara verificada pero diminuta o de perfil produce AUs basura. Dos cortes simples:

```
def calidad_ok(bbox, frame_shape, min_ancho=100):
    top, right, bottom, left = bbox
    ancho = right - left
    alto = bottom - top
    # 2a) Tamaño: cara suficientemente grande
    if ancho < min_ancho:          # 100 px mínimo; 150 ideal
        return False
    # 2b) Proporción (proxy de frontalidad): una cara muy "aplastada"
    #     suele indicar perfil fuerte o detección parcial
    if not (0.6 <= ancho / max(alto, 1) <= 1.1):
        return False
    return True
```

- **min_ancho = 100 px** es el umbral por debajo del cual OpenFace/MediaPipe pierden fiabilidad. En 360p esto descarta casi todos los planos abiertos — y está bien que lo haga.
- Si tienes **head pose** (OpenFace da **pose_Ry**), un filtro mejor que el proxy es **|yaw| ≤ 30°**.

Nota de resolución: este filtro es el que más recorta en 360p. Documenta **qué porcentaje de frames se descarta por tamaño**. Si es muy alto, vale la pena conseguir el video en HD (las audiencias JEP se archivan en mejor resolución) — es la mejora individual de mayor impacto, pero **no es requisito** para esta versión.

5. Filtro 3 — Identidad de voz (verificación manual)

Como son **solo 5 subcasos**, no hace falta automatizar nada: verifica a mano qué hablante de la diarización es el compareciente.

5.1 Procedimiento (una vez por subcaso)

1. Abre la transcripción (**corpus_c/<subcaso>.txt**) y el ***_diarization.json**.
2. Localiza dónde habla el compareciente por **contenido**: primera persona, reconocimiento de responsabilidad, referencias a su unidad/rango.
3. Mira a qué **SPEAKER_XX** corresponde ese contenido en la diarización.
4. Si la diarización lo **fragmentó** en varios SPEAKER (ej. SPEAKER_01 y SPEAKER_03 son la misma persona), **anótalos todos** como «compareciente».
5. Registra el mapeo en una tabla:

Subcaso	SPEAKER(s) del compareciente	Verificado por
Casanare	SPEAKER_03	(revisión manual)
Catatumbo	SPEAKER_01 (+ SPEAKER_03?)	(revisión manual)
...	...	...

Esto reemplaza la heurística «el que más habla» por una **identificación verificada**, y de paso corrige la sobre-segmentación consolidando los fragmentos. Es una tarde de trabajo para los 5 casos.

(Opcional, solo si quisieras automatizarlo: un embedding de voz con `resemblyzer` enrolado desde un segmento confirmado. No es necesario para esta versión.)

6. Juntar todo: construcción de la muestra

Pseudocódigo del flujo completo por subcaso:

```

turnos_validos = [] # cada elemento = una intervención del compareciente

for turno in segmentos_diarizacion:
    if turno["speaker"] not in SPEAKERS_COMPARECIENTE[subcaso]:
        continue # Filtro 3: solo el compareciente
    if (turno["end"] - turno["start"]) < 3.0:
        continue # turnos demasiado cortos no aportan

    # --- ROSTRO: muestrear frames del turno a ~5 fps ---
    aus_frames = []
    for t in arange(turno["start"], turno["end"], 0.2): # cada 0.2 s = 5 fps
        frame = leer_frame(video, t)
        bbox, es_el = cara_del_compareciente(frame) # Filtro 1: identidad
        if not es_el:
            continue
        if not calidad_ok(bbox, frame.shape): # Filtro 2: calidad
            continue
        aus_frames.append(extraer_AUs(frame, bbox)) # MediaPipe/OpenFace

    cobertura = len(aus_frames) / n_frames_del_turno
    if cobertura < 0.40: # turno con muy pocas caras válidas → descartar
        continue

    # --- AGREGACIÓN por turno: MEDIANA, no media ---
    aus_turno = np.median(np.vstack(aus_frames), axis=0)

    # --- VOZ: eGeMAPS del audio del turno ---
    egemaps_turno = extraer_egemaps(audio[turno["start"]:turno["end"]])

    turnos_validos.append({"start": turno["start"], "end": turno["end"],
                          "aus": aus_turno, "egemaps": egemaps_turno,
                          "n_frames_validos": len(aus_frames)})

```

El ICM del subcaso se calcula sobre `turnos_validos` (la mediana/distribución de los turnos), no sobre frames sueltos.

Puntos que esto corrige respecto al código actual:

- Ya no entra ninguna cara que no sea el compareciente (Filtro 1).
- Ya no entran caras-ruido de planos abiertos (Filtro 2).
- Ya no se confía en «el que más habla» (Filtro 3).
- La unidad es el turno, agregado con mediana → resistente a outliers y estadísticamente correcto.

7. Qué reportar (cobertura y honestidad)

Por cada subcaso, reportar en una tabla:

- **Nº de turnos válidos** del compareciente.
- **% de frames que pasan los filtros** (identidad + calidad) — esta es la tasa de detección real, con su denominador claro («frames de turnos del compareciente»).
- **Turnos descartados** por baja cobertura (<40%).

Y declarar como limitación, sin dramatismo:

- La resolución (360p) limita el tamaño de cara y por tanto la cobertura facial; subir a HD la mejoraría.
- Con 5 subcasos, los resultados son **descriptivos/exploratorios**, no inferencia poblacional.

8. Texto sugerido para el capítulo de metodología

«La extracción de features faciales y prosódicas se restringió al compareciente mediante un procedimiento de verificación de identidad. Para cada subcaso se enroló una imagen de referencia del compareciente y se aceptaron únicamente los rostros cuyo descriptor facial (face_recognition, 128-d) coincidía con la referencia bajo un umbral de distancia de 0.55, y que superaban un mínimo de calidad (ancho de rostro ≥ 100 px, vista cuasi-frontal). El hablante objetivo se verificó manualmente sobre la diarización para cada audiencia. La unidad de análisis es la intervención (turno) del compareciente; las Action Units se agregaron por turno mediante la mediana de los frames válidos, y la prosodia mediante los functionals eGeMAPS del segmento. Se reporta la cobertura (porcentaje de frames válidos) por subcaso; dada la resolución de archivo (360p) y el número de subcasos (n=5), los resultados multimodales se interpretan de forma exploratoria.»

9. Mejoras futuras opcionales (NO requeridas para esta versión)

Solo si en el futuro se quisiera llevar la Capa 3 a estándar de publicación:

- **Active speaker detection automática** (TalkNet/Light-ASD) para confirmar por frame que el compareciente está hablando.
- **OCR del banner (lower-third)** para anclar identidad+rol automáticamente.

- **Estratificación por fase** del acto (apertura / relato / reconocimiento / disculpa / interpelación) → permite comparar congruencia entre fases.
- **Inferencia multinivel + bootstrap** (frames $\subset$ turnos $\subset$ sujeto) para intervalos de confianza.
- **Validación perceptual** del ICM con jueces humanos (Baird & Coutinho 2019).

Estas son líneas de robustecimiento, no condiciones de la tesis.

Checklist operativo

- [] Una foto de referencia `ref_<subcaso>.jpg` por compareciente (frontal, nítida).
- [] Tabla de mapeo subcaso → SPEAKER(s) del compareciente (verificación manual).
- [] Filtro 1 (identidad facial, tolerancia 0.55) implementado.
- [] Filtro 2 (tamaño ≥ 100 px + frontalidad) implementado.
- [] Extracción de AUs solo en frames que pasan ambos filtros, a ~ 5 fps.
- [] Agregación por turno con **mediana** (no «primer frame», no media).
- [] eGeMAPS por turno del SPEAKER verificado.
- [] Tabla de cobertura (turnos válidos + % frames válidos) por subcaso.
- [] Limitaciones declaradas (360p, n=5, exploratorio).